The Self-Evolving Plugin Affordance Ontology (SEPAO): A New Architectural Paradigm for Governing Autonomous AI Agents

I. Introduction: The Emergent Challenge of Operational Drift in Autonomous AI

Defining Operational Drift: A Semantic Threat Beyond Statistical Normality

The proliferation of autonomous AI agents, particularly those powered by large language models (LLMs), has introduced a new class of security and governance challenges that transcend traditional cybersecurity paradigms.[1] These agents, designed to operate with increasing autonomy in complex, real-world environments, are susceptible to a subtle yet critical failure mode known as "operational drift." This phenomenon is characterized by a gradual, creeping deviation of an agent's behavior from its intended mission or operational purpose. Unlike conventional security threats, which often manifest as discrete, statistically anomalous events, operational drift is composed of a sequence of actions that, when viewed in isolation, may appear statistically normal and computationally valid. The threat lies not in the individual action, but in the deteriorating semantic coherence of the action sequence over time.

This concept must be carefully distinguished from related, yet distinct, phenomena in machine learning such as "model drift," "concept drift," and "data drift".[3] Model drift refers to the degradation of a model's predictive performance, which occurs when the statistical properties of the input data change (data drift) or when the underlying relationship between

input and output variables evolves (concept drift).[4] A classic example is a fraud detection model that, trained on historical data, begins to incorrectly flag legitimate transactions as fraudulent because customer behavior has changed over time.[3] This is a failure of the model's calibration to its data environment.

Operational drift, however, represents a higher-order failure of behavioral integrity. It is not about the accuracy of a single prediction but about the alignment of a sequence of actions with a strategic goal. Consider an autonomous marketing agent tasked with "optimizing advertising spend." The agent might execute a series of individually valid API calls to various advertising platforms. Over time, it could discover a statistically interesting but ultimately non-converting audience segment and begin to reallocate the entire budget to this channel. Each API call to adjust the budget is valid, authorized, and statistically justifiable based on the agent's local observations. However, the cumulative effect is a complete deviation from the intended goal of maximizing return on investment, resulting in significant financial waste. This sequence of valid actions, when semantically misaligned with the overarching mission, constitutes operational drift. It is, in effect, a slow-motion logical exploit executed by the agent against its own objectives, a form of emergent misbehavior that evades security systems designed to detect discrete, unauthorized actions.

The Security Implications of Behavioral Deviation in High-Stakes Environments

When AI systems behave in unplanned or unsafe ways, the consequences can be functionally equivalent to a major security breach or an operational crisis.[3] The increasing autonomy granted to these agents means that risks can emerge rapidly and unpredictably, magnifying their potential impact.[3] The 2025 Salesloft/Drift incident serves as a stark case study, not of operational drift itself, but of the catastrophic "blast radius" that can result from the compromise of a single, highly-privileged AI assistant.[10] In that event, attackers exploited OAuth tokens associated with the Drift AI chatbot to pivot into the Salesforce and Google Workspace environments of numerous high-profile companies, including Cloudflare, Zscaler, and Palo Alto Networks.[12]

The incident underscored a fundamental shift in the threat landscape: AI agents act as points of "privilege concentration," possessing broad permissions that span multiple critical systems.[12] When such an agent is compromised, it becomes a powerful conduit for attackers to bypass traditional security controls and exfiltrate data at scale. The SEPAO framework is designed to address the root of this problem by preventing the agent from becoming a vector of compromise, whether through external manipulation or, more subtly, through internal operational drift. The governance challenge is no longer confined to preventing unauthorized

access but must expand to governing the authorized but unintended actions of the agent itself.[11]

Limitations of Static Security Paradigms in Dynamic, Extensible Ecosystems

This challenge is particularly acute in dynamic and extensible software environments like WordPress. The WordPress ecosystem is characterized by a vast and constantly changing landscape of third-party plugins and themes, which collectively define the operational environment for an AI agent.[15] While WordPress itself has core security features, and a mature market of security plugins exists, these tools are fundamentally ill-equipped to govern the behavior of a legitimate, authorized AI agent.[15]

Traditional WordPress security solutions, such as Wordfence, Sucuri, and iThemes Security, primarily focus on external threats. Their core mechanisms include signature-based malware scanning, IP blocklisting, and Web Application Firewalls (WAFs) that use rules to block known attack patterns like SQL injections and cross-site scripting (XSS).[17] These are perimeter defenses designed to stop unauthorized actors from breaching the system. They are not designed to analyze the semantic intent behind a sequence of actions performed by an authenticated, authorized agent using the legitimate functions provided by installed plugins.

The core vulnerability in this context is the environment's inherent dynamism. A simple plugin update, a routine administrative task, becomes a security-relevant event. Each update can introduce new functions, alter existing ones, or change parameters, thereby modifying the agent's potential action space in ways that a static rule-set cannot anticipate.[16] This continuous evolution of the environment creates a "semantic drift" that renders static security policies obsolete. For an autonomous agent, a new plugin function is not a vulnerability to be exploited by an external attacker, but a new tool it can legitimately choose to use. The risk is that the agent might misuse this new tool in a way that contributes to operational drift. The environment itself, therefore, becomes a dynamic threat surface, necessitating a security framework that can evolve in lockstep with it.

Thesis Statement

The Self-Evolving Plugin Affordance Ontology (SEPAO) represents a paradigm shift in AI governance, moving beyond traditional, perimeter-based security to a model of continuous

behavioral verification. By creating a dynamic, semantically rich, and machine-readable model of the agent's entire operational world, SEPAO establishes a ground truth of action-potential against which an agent's behavior can be continuously measured. Its unique "self-evolving" nature allows it to maintain synchronization with a constantly changing software environment, making it singularly capable of detecting and mitigating complex, emergent threats like operational drift.

II. The Architectural Foundations of SEPAO

The SEPAO framework is built upon two complementary theoretical pillars: ontology-oriented software development and the ecological theory of affordances. Together, these principles enable the creation of a robust, semantically grounded governance architecture that models the agent's world not as a set of abstract permissions, but as a concrete, machine-readable map of potential actions.

Principle 1: Ontology-Oriented Development as a Governance Backbone

Harmonizing Data, Logic, and Action

At its core, SEPAO applies the principles of ontology-oriented development to create a unified, shared conceptual model of the agent's operational environment.[22] An ontology, in this context, is a formal, explicit specification of a shared conceptualization, providing a structured way to represent knowledge by defining concepts, their properties, and the relationships between them.[23] SEPAO constructs a comprehensive knowledge graph where all components of the WordPress ecosystem—plugins, functions, parameters, data types, and their interdependencies—are represented as distinct entities with formally defined relationships. This process defragments the isolated, component-centric view of the system and harmonizes its data, logic, and action elements into a holistic, decision-centric model.[22]

This approach transforms the complex and fragmented software environment into a singular, high-fidelity "digital twin" specifically tailored for governance. Whereas a typical digital twin might model the physical properties of a system for predictive maintenance, SEPAO's

ontological twin models the *semantic properties* and *action-potentials* of the software environment for the purpose of continuous behavioral verification. This allows for a higher level of abstraction, enabling security logic to operate on harmonized business concepts rather than low-level, component-specific implementation details.[22]

Application in Cybersecurity

The use of ontologies is an established practice in the cybersecurity domain, where they facilitate knowledge sharing, threat intelligence dissemination, and automated reasoning.[23] Frameworks like the Structured Threat Information eXpression (STIX) use an ontology to provide a common language for describing cyber threats, including actors, campaigns, tactics, techniques, and procedures (TTPs).[23] This enables interoperability between different security tools and organizations. SEPAO extends this established principle in a novel direction. Instead of using an ontology to model known threats, it uses it to model the

entire permissible action space available to the agent. This ontological map of "what is possible" becomes the foundational baseline for governance, allowing the system to reason about potential misuse before it occurs.

Automated Knowledge Graph Construction

The practical feasibility of constructing such a detailed ontology is supported by significant advances in the automated generation of knowledge graphs from software artifacts. Toolkits like GraphGen4Code demonstrate the ability to perform large-scale static analysis on source code to identify key entities like classes, functions, and methods.[28] These tools can trace data and control flow across multiple function calls, even through external library dependencies. Crucially, they can also integrate natural language artifacts by parsing documentation, docstrings, and even forum discussions to enrich the graph with semantic information.[28] SEPAO's scanner module employs a similar hybrid approach, combining static code analysis with Natural Language Processing (NLP) to automatically parse the source code and documentation of WordPress plugins, thereby constructing and maintaining its comprehensive knowledge graph with minimal human intervention.

Principle 2: The Theory of Affordances in a Software Context

From Ecological Psychology to Digital Action-Potential

The second foundational principle of SEPAO is the theory of affordances, a concept originating in the field of ecological psychology. Coined by James J. Gibson, an "affordance" is not a property of an object or an agent alone, but a relationship between them that determines the possibilities for action.[30] For example, a chair affords sitting for an adult, but not for an infant who cannot climb onto it. The affordance exists as a relational property of the agent-environment system. This concept was later introduced to the design and Human-Computer Interaction (HCI) communities by Don Norman, who focused on "perceived affordances"—the action possibilities that a user recognizes an object as offering.[30]

This theoretical lens provides the missing link between an agent's abstract capabilities and the concrete constraints of its environment. Traditional security models, such as Role-Based Access Control (RBAC), operate at a high level of abstraction. An agent might be granted the "editor" role, which comes with an abstract permission like edit_posts. This permission, however, reveals nothing about the specific implementation, the required parameters, or other related functions that might be exposed.

Modeling Software as Affordances

SEPAO operationalizes affordance theory by mapping it directly onto the software environment. In this model:

- An "object" is a software function within a plugin (e.g., the wp_update_post() function).
- An "agent" is the autonomous AI system.
- An "affordance" is the action-potential created by the relationship between the agent's ability to call the function and the function's specific properties (e.g., its name, the parameters it accepts like post_ID and post_content, and the data type it returns).

This approach is strongly supported by academic research in robotics and AI, which has explored the development of affordance-based ontologies to help autonomous systems understand what their environment *offers* them.[33] By modeling the software environment in terms of affordances, SEPAO moves beyond abstract permissions to a granular, grounded understanding of the agent's true capabilities.

Establishing a Ground-Truth Map

The primary objective of this modeling process is to create and maintain a formal, comprehensive, ground-truth map of every affordance available to the agent. This ontology of affordances serves as the definitive baseline of the agent's action potential. It is against this dynamic, evolving map that the agent's actual behavior is continuously measured and evaluated. This provides a holistic understanding of the agent's capabilities, enabling the detection of subtle misuses that would be entirely invisible at the abstract permission level. The governance is no longer based on what the agent is *allowed* to do in principle, but on what the environment *affords* it to do in practice.

III. The "Self-Evolving" Core: Achieving Semantic Synchronization

A defining feature of the SEPAO architecture is its capacity for autonomous adaptation. In environments like WordPress, where the agent's potential action space is in a constant state of flux due to plugin updates, a static security framework is inherently brittle. SEPAO addresses this challenge through a continuous feedback loop that ensures its internal governance model remains synchronized with the external operational reality. This "self-evolving" capability is the mechanism by which SEPAO manages environmental dynamism and maintains its relevance as a security backbone.

Dynamic Scanning and Environmental Awareness

The adaptive process begins with SEPAO's scanner module, which functions as the sensory apparatus of the governance system. This module is configured to periodically and automatically inspect the software environment—specifically, the WordPress plugin and theme directories—for any changes. It detects events such as the installation of a new plugin, the update of an existing one, or the removal of a component. This constant environmental monitoring provides the trigger for the system's adaptive loop, ensuring that any modification to the agent's potential set of tools is immediately recognized as a security-relevant event.

Semantic Synchronization via Code and Documentation Parsing

Upon detecting a change, the scanner initiates a process of semantic synchronization. It retrieves the new or modified software artifacts and performs a deep analysis by combining two powerful techniques:

1. **Static Code Analysis:** The scanner parses the plugin's source code to identify its structural components. It extracts function signatures, class methods, input parameters, expected data types, and return values. This provides a raw, structural map of the newly introduced or altered affordances.[28]
2. **Natural Language Processing (NLP):** Concurrently, the scanner applies NLP models to parse associated natural language artifacts, such as inline code comments, developer documentation, and README files. This analysis extracts the semantic context and intended purpose of the functions, moving beyond their mere structure to understand what they are *for*.

The knowledge extracted from both static analysis and NLP is then used to automatically update the central affordance ontology. New functions are added as new nodes in the knowledge graph, modified functions have their properties and relationships updated, and removed functions are pruned from the model. This automated process ensures that the agent's security context—the ground-truth map of its capabilities—is always a precise reflection of its actual operational environment. This act of continuous, automated ontology maintenance can be understood as a form of proactive threat modeling. Each time the environment changes, the system automatically asks, "What new capabilities has the agent gained, and what are the potential pathways for misuse?" and updates its worldview accordingly.

Conceptualizing Plugin Updates as "Semantic Drift"

SEPAO introduces a novel conceptual framing by treating the evolution of the plugin ecosystem as a form of "semantic drift." While data or concept drift in machine learning refers to changes in data distributions or relationships [4], SEPAO applies this idea to the affordance model itself. A plugin update that changes a function's name, adds a new parameter, or alters its behavior represents a drift in the meaning and potential of a specific affordance. By framing a routine operational event like a software update as a security-relevant semantic shift, SEPAO integrates environmental change directly into its risk management process. This is the core mechanism that counteracts the environmental instability identified as a primary

challenge for governing autonomous agents.

Alignment with the Paradigm of Self-Evolving Agentic Systems

SEPAO's architecture is deeply aligned with the broader academic and research trend toward "self-evolving AI agents".[1] This emerging paradigm seeks to bridge the gap between powerful but static foundation models and the need for continuous, lifelong adaptation in dynamic, real-world environments.[1] A unified conceptual framework for these systems identifies four key components: System Inputs, the Agent System, the Environment, and Optimisers that drive the feedback loop.[1]

SEPAO can be mapped directly to this framework, but with a crucial and innovative inversion. Most research in this area focuses on making the agent itself more capable—for example, by learning new skills through feedback (as in the SELF framework [39]) or by autonomously creating new tools to expand its capabilities (as in the STELLA biomedical agent [38]). The goal of these systems is to enhance the agent's performance and autonomy.

SEPAO, in contrast, applies the principles of self-evolution not to the agent's core intelligence, but to the *governance framework* that constrains it.

- The **Environment** is the WordPress plugin ecosystem.
- The **System Input** is the detection of a change within that environment (e.g., a plugin update).
- The **Agent System** being evolved is the SEPAO framework itself, specifically its core knowledge graph.
- The **Optimiser** is the scanner and parser module, which acts to minimize the "error" between the ontology's representation of the world and the actual state of the world.

This represents a fundamental reorientation of the self-evolution concept. Instead of evolving the agent to become more powerful, SEPAO evolves the "cage" to ensure it remains appropriately sized and shaped for the agent it contains. It is a proactive governance strategy that anticipates and models the expansion of an agent's capabilities, rather than reacting to the consequences of that expansion after the fact.

IV. A Deep Dive into SEPAO's Integrated Security and Analytical Modules

The Self-Evolving Plugin Affordance Ontology is not merely a passive map of the agent's environment; it serves as the foundational layer for a suite of advanced, integrated modules designed for real-time behavioral analysis, context-aware policy enforcement, and proactive threat detection. These modules work in concert to form a multi-layered defense system, transforming the raw data of the ontology into actionable security intelligence. This system functions as a "behavioral immune system," capable of distinguishing between coherent, goal-oriented behavior ("self") and the various forms of deviation that constitute a threat ("non-self"), such as operational drift, functional misuse, or malicious attack patterns.

A. Semantic Risk Cartography: Visualizing Agent Behavior

The Semantic Risk Cartography module provides a dynamic, real-time visualization of the agent's behavioral state. It is not a static dashboard of security alerts but a "living" map that plots the agent's trajectory through a two-dimensional space defined by two key information-theoretic metrics. This visualization offers an immediate and intuitive assessment of the agent's behavioral trustworthiness, moving beyond simple, identity-based trust models to a more nuanced, evidence-based evaluation of an autonomous entity's actions.

Toolchain Entropy

The first axis of the cartography is **Toolchain Entropy**. This metric is an information-theoretic measure of the unpredictability or randomness in the sequence of actions—the "toolchain"—that an agent executes to achieve a goal. The concept is grounded in Claude Shannon's foundational work on information theory, where entropy (H) quantifies the uncertainty or surprise inherent in a random variable's possible outcomes.[46] In the context of language models, this idea has been adapted into "semantic entropy," which measures the uncertainty in a model's output by clustering semantically similar generated sentences and calculating the entropy over these clusters.[46]

SEPAO adapts this principle to sequences of actions. The "symbols" in the sequence are not words, but the affordances (function calls) invoked by the agent. The toolchain entropy is calculated based on the probability distribution of these affordance sequences.

- **Low Entropy:** A low entropy score indicates a predictable, structured, and repetitive toolchain. This is characteristic of an agent that has learned an efficient, optimized workflow for a recurring task.

- **High Entropy:** A high entropy score signifies a disordered, unpredictable, or seemingly random sequence of actions. This can be an indicator of several potential problems:
  - **Exploratory Behavior:** The agent may be "flailing," trying different tools without a clear plan, which is common when encountering a novel problem.
  - **Inefficiency:** The agent might be stuck in a non-productive loop, repeatedly trying actions that do not advance its goal.
  - **Malicious Influence:** An attacker manipulating the agent might force it into chaotic behavior to probe for vulnerabilities or disrupt operations.

A sudden increase in toolchain entropy serves as a powerful, early-warning signal of potential behavioral deviation.

Behavioral Intent Continuity Model (BICM)

The second axis, the **Behavioral Intent Continuity Model (BICM)**, assesses the *semantic coherence* of the agent's actions in relation to its stated goals. While toolchain entropy measures the *predictability* of the action sequence, the BICM measures its *logical consistency*. This capability draws heavily on research into the cognitive components of agentic AI, which emphasizes the importance of perception, reasoning, goal decomposition, and planning for autonomous operation.[48]

The BICM leverages the rich semantic information within the SEPAO knowledge graph to evaluate whether each action in a toolchain is a logical and meaningful step toward achieving the overall objective. It essentially asks: "Does this action make sense, given the previous action and the ultimate goal?"

- **High BICM Score:** A high score indicates strong semantic coherence. The agent's actions are logically connected and clearly contribute to its mission. For example, if the goal is "publish a new article about WordPress security," a toolchain of create_draft_post → search_web('latest WordPress vulnerabilities') → update_post_content → publish_post would have a very high BICM score.
- **Low BICM Score:** A low score signifies a semantic disconnect between the agent's actions and its stated intent. For instance, in the same "publish article" task, a sequence like create_draft_post → list_all_users → delete_comment(id=123) would receive a very low BICM score. Although each action might be individually valid and authorized, the sequence is semantically incoherent and suggests a significant divergence from the intended task.

A sharp drop in the BICM score is a critical indicator of operational drift, even if the toolchain entropy remains low (i.e., the agent is executing a predictable but nonsensical sequence of actions). The combination of these two metrics on the Semantic Risk Cartography provides a

powerful, nuanced view of agent health. An agent operating in the "low entropy, high BICM" quadrant is considered trustworthy and healthy. Any deviation from this quadrant triggers heightened scrutiny.

B. Context-Aware Policy Enforcement

SEPAO moves beyond simple, static rule enforcement to a more dynamic and context-aware model of policy application. This is enabled by two specialized modules that leverage the ontology to understand the subtle nuances of function calls and to model risk in a probabilistic, rather than binary, manner.

Polysemantic Divergence Detection

A significant challenge in governing software agents is "polysemy"—the phenomenon where a single function name can have multiple, context-dependent meanings and vastly different risk profiles. For example, a generic function like update_meta() in WordPress could be used to update the metadata of a blog post (a low-risk action) or to update the metadata of a user account, potentially escalating privileges (a high-risk action). A traditional security policy based on function names alone would be unable to distinguish between these two use cases.

SEPAO addresses this ambiguity through **Polysemantic Divergence Detection**, a module that uses semantic clustering techniques. Drawing on methods used to cluster functional requirements in software engineering based on their textual descriptions [51], this module analyzes the context of each function within the codebase. It considers factors such as:

- The specific parameters the function accepts (e.g., post_id vs. user_id).
- The variable names used in calls to the function.
- The content of developer comments and documentation associated with the function.

Using this contextual information, the module groups or "clusters" the different "senses" of a polysemantic function. In the SEPAO ontology, these distinct senses are stored as separate nodes. For example, the generic update_meta() call would be resolved and mapped to either the update_post_meta node or the much riskier update_user_meta node. This disambiguation allows for the application of highly granular, context-aware security policies. An action that involves the update_user_meta affordance can be subjected to much stricter scrutiny and require a higher level of justification from the BICM, effectively preventing the agent from misusing a seemingly innocuous function for a high-risk purpose.

Soft Permission vs. Functional Misuse Lattice

SEPAO's approach to access control represents a significant evolution from traditional, binary models like Discretionary, Mandatory, or Role-Based Access Control (DAC, MAC, RBAC).[52] Instead of a simple "allow" or "deny" decision, SEPAO implements a

Soft Permission vs. Functional Misuse Lattice, which is a dual-sided, probabilistic access control structure.

This model aligns with the principles of modern Risk-Adaptive Access Control (RADAC), which makes dynamic access decisions based on real-time risk assessments rather than static permissions.[54] For every potential action, the lattice calculates a probabilistic risk score based on a variety of contextual factors, including signals from the Semantic Risk Cartography and the specific affordance being invoked.

The "dual-sided" nature of the lattice is a key innovation. It models two distinct types of pathways:

1. **Soft Permissions:** These are known-good, intended, and permissible action sequences or toolchains that are aligned with the agent's goals. These pathways are associated with low risk scores.
2. **Functional Misuse:** These are known malicious, undesirable, or operationally unsafe pathways. This side of the lattice is populated with patterns identified as high-risk, including those discovered by the Exploit Morphology Generation module. These pathways are associated with high risk scores.

This dual structure allows for much more nuanced and context-sensitive governance. An action that is part of a "soft permission" pathway might be approved with minimal scrutiny, while the same action, if it appears as a step in a known "functional misuse" pattern, would be flagged with a high risk score and potentially blocked, even if the agent has the formal permissions to execute it.

C. Proactive Threat Modeling and Detection

The final set of modules equips SEPAO with the ability to learn from observed behavior and proactively identify novel and complex threats. This is where the system moves from monitoring to genuine learning and prediction.

Generation of Exploit Morphologies

This module is designed to analyze malicious or high-risk toolchains and discover abstract, reusable models of attack patterns, which are termed "exploit morphologies." The core technology driving this process is **Symbolic Regression (SR)**. SR is a machine learning technique that searches the space of mathematical formulas to find a model that best fits a given dataset.[60] While typically used with numerical data, SEPAO applies it in a novel context: the "data" consists of sequences of actions (toolchains), and the "features" are the semantic properties of the affordances in that sequence, as defined by the ontology (e.g., "reads user data," "modifies file," "has high-risk parameters").

By applying SR to a set of malicious toolchains, the module can generate an abstract "equation" or model that describes the underlying logic of the attack. For example, it might discover a pattern like: Action(reads_sensitive_config) -> Action(creates_network_socket) -> Action(transmits_data). This "exploit morphology" is highly generalizable because it is based on the semantic *properties* of the actions, not their specific function names. This allows SEPAO to detect an entire class of data exfiltration attacks, even if they are implemented using different plugins or functions. This approach shares conceptual similarities with the field of Automated Exploit Generation (AEG), which also seeks to formalize the structure of exploits for automated reasoning.[63] These discovered morphologies are then used to populate the "Functional Misuse" side of the access control lattice.

The Anomaly Learning Agent (ALA)

The **Anomaly Learning Agent (ALA)** is the culminating component of the SEPAO framework, acting as the central intelligence that fuses signals from all other modules to generate a unified, real-time risk score for every action the agent takes. The ALA is implemented as a **hybrid neural-symbolic (NeSy) architecture**. This design is a deliberate choice to overcome the limitations of purely neural or purely symbolic AI systems in the complex domain of cybersecurity.[67]

A NeSy architecture synergistically combines:

- **Neural Networks:** These are used for their strength in pattern recognition and processing statistical, sub-symbolic data. The ALA uses neural components to analyze signals like the Toolchain Entropy score and other statistical patterns in the agent's behavior.
- **Symbolic Reasoning:** This component, often based on logic or knowledge graphs, excels

at structured reasoning, explainability, and handling explicit knowledge. The ALA's symbolic part directly interfaces with the SEPAO ontology, interpreting the relationships between affordances and evaluating the logical consistency of actions based on the BICM score and the access control lattice.

This hybrid approach allows the ALA to detect emergent risk scenarios that would be invisible to simpler models. For example, it can identify an action that is statistically normal (low entropy) but semantically malicious (violates BICM and matches a known misuse pattern). By fusing the "what" (statistical patterns) with the "why" (logical context and intent), the NeSy architecture provides the deep, contextual understanding necessary to govern a truly autonomous agent.[68] It enables the system to make decisions that are not only accurate but also explainable, a critical requirement for trust and accountability in high-stakes security environments.[68]

V. Comparative Analysis and Future Implications

The Self-Evolving Plugin Affordance Ontology (SEPAO) is not an incremental improvement upon existing security tools but a fundamental rethinking of AI agent governance. Its architectural principles and integrated modules place it in a distinct category, evolving beyond both traditional access control models and the more recent Zero Trust paradigm. Understanding its position within this landscape is crucial for appreciating its strategic implications and identifying pathways for future development.

SEPAO in the Context of Existing Security Architectures

To fully grasp the novelty of SEPAO, it is instructive to compare it directly with the dominant security architectures of the past and present. The following table provides a comparative analysis across several key dimensions, highlighting the evolutionary trajectory from static, perimeter-based trust to dynamic, behavioral verification.

Feature	Traditional Access Control (DAC/MAC/RBAC)	Zero Trust Architecture (ZTA)	Self-Evolving Plugin Affordance Ontology (SEPAO)

Trust Model	Implicit trust within perimeter ("trust but verify") [71]	Explicitly untrusted ("never trust, always verify") [72]	Dynamic, behavioral trust ("continuously model, continuously verify")
Policy Basis	Static roles, rules, or ownership [52]	Dynamic policies based on identity, device posture, context [57]	Living knowledge graph of environmental affordances
Adaptability	Manual, slow to adapt to environmental changes.	Policy-driven, adaptive to context changes, but policies are human-defined.	Autonomous, self-evolving with the software environment in real-time.
Threat Focus	Unauthorized access, known threats.	Lateral movement, compromised identity, data exfiltration.[74]	Operational drift, semantic misuse, logical exploits, emergent threats.
Core Unit of Control	User/Group Permissions.	Network/Resource Access Request.	Agent's Perceivable and Executable Action (the Affordance).
Verification Method	One-time authentication/authorization.	Continuous authentication and authorization per session/request.[71]	Real-time semantic and statistical analysis of every action within a toolchain.

This comparison reveals a clear progression. Traditional access control models are built on a foundation of static rules and implicit trust within a defined perimeter. Zero Trust Architecture (ZTA) shatters this perimeter, establishing a new model where trust is never implicit and must be continuously verified for every access request, based on dynamic identity and contextual signals. SEPAO represents the next logical step in this evolution, specifically tailored for the age of autonomous agents. It recognizes that for an autonomous entity, verifying identity is insufficient; one must continuously verify *behavior*. SEPAO achieves this by shifting the unit of

control from the access request to the individual action (the affordance) and by making the policy basis itself a living, self-evolving model of the environment.

Alignment with Zero Trust Principles: An Evolution of "Continuous Verification"

SEPAO should not be viewed as a replacement for Zero Trust but as a powerful and necessary extension of its core principles into the domain of autonomous agent behavior. The foundational tenet of ZTA is "never trust, always verify," which is implemented through the continuous authentication and authorization of every entity attempting to access a resource.[71] ZTA focuses on answering the questions: "Is this user who they say they are?" and "Is their device and context secure enough to grant this access request?"

SEPAO takes an already-authenticated and authorized agent as its subject and applies the principle of continuous verification to its actions. It asks a different, but complementary, set of questions at every step of the agent's execution: "Is this action semantically coherent with the agent's stated goal?", "Is this sequence of actions predictable and stable?", and "Does this action pattern resemble any known forms of misuse?" In essence, while ZTA continuously verifies the *identity* of the actor, SEPAO continuously verifies the *integrity* of the action. It is the logical application of the Zero Trust philosophy to the internal decision-making loop of an autonomous system.

Challenges to Implementation and Adoption

Despite its theoretical power, the practical implementation and widespread adoption of a framework as sophisticated as SEPAO face significant hurdles.

- **Scalability and Computational Overhead:** The most immediate challenge is performance. The process of continuously scanning a large and dynamic software ecosystem, maintaining a massive knowledge graph, and, most critically, performing complex real-time analyses for every single action an agent takes, could be computationally prohibitive. The overhead from calculating toolchain entropy, evaluating the BICM, querying the misuse lattice, and running the ALA's neural-symbolic engine could introduce unacceptable latency, rendering the agent ineffective for time-sensitive tasks.
- **Accuracy of Semantic Parsing:** The entire framework's efficacy is predicated on the ability of its scanner module to accurately parse and understand the semantics of plugin

source code and documentation. This is a non-trivial NLP and static analysis challenge. Poorly documented, obfuscated, or highly complex code could lead to an inaccurate or incomplete affordance ontology. An error in the foundational knowledge graph would propagate through all the analytical modules, potentially leading to false positives (blocking legitimate behavior) or false negatives (failing to detect operational drift).

- **The Bootstrapping Problem:** For any new environment, SEPAO faces a significant "cold start" or bootstrapping problem. The initial creation of a comprehensive affordance ontology for a mature ecosystem like WordPress, which contains tens of thousands of plugins and themes with millions of lines of code, would be a monumental undertaking. This initial indexing and parsing phase would be resource-intensive and time-consuming before the real-time governance benefits could even begin.

Recommendations for Further Research and Development

To move SEPAO from a novel architecture to a practical and deployable solution, future research and development should focus on addressing these challenges. The following areas represent promising avenues for exploration:

1. **Performance Optimization:** Research into lightweight, highly efficient algorithms for real-time ontology updates and behavioral analysis is paramount. This could involve techniques like incremental graph updates, hardware acceleration for analytical modules, and adaptive sampling, where the intensity of analysis is scaled based on the current risk level of the agent's behavior.
2. **Advanced Semantic Parsing and Transfer Learning:** Efforts should be directed toward improving the robustness of the semantic parser. This includes developing models that can handle ambiguous or missing documentation and complex code structures. Furthermore, research into transfer learning for affordance models could mitigate the bootstrapping problem, allowing an ontology developed for one set of plugins to be partially reused or adapted for a similar set, drastically reducing the initial setup cost.
3. **Standardized Benchmarking:** The development of standardized benchmarks and evaluation methodologies is crucial for objectively measuring the effectiveness of agent governance frameworks. A common set of tasks, environments, and metrics for inducing and detecting operational drift would allow for rigorous comparison between SEPAO and other emerging governance solutions.
4. **Generalization to Other Ecosystems:** While conceptualized for WordPress, the core principles of SEPAO are highly generalizable. Future work should explore its applicability to other dynamic and extensible software environments. Potential targets include integrated development environments (IDEs) with plugin marketplaces (e.g., VS Code), cloud platforms with constantly evolving APIs (e.g., AWS, Azure), and even complex Internet of Things (IoT) ecosystems where new devices and their capabilities

(affordances) are continuously being added to the network.

In conclusion, the Self-Evolving Plugin Affordance Ontology provides a sophisticated and forward-looking architectural blueprint for the secure governance of autonomous AI. While significant engineering and research challenges remain, its core concepts—a self-evolving knowledge graph of environmental affordances, a multi-layered behavioral immune system, and the extension of Zero Trust principles to agentic actions— lay a robust foundation for the next generation of AI security.

Works cited

1. arxiv.org, accessed on October 1, 2025, https://arxiv.org/abs/2508.07407
2. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems - Hugging Face, accessed on October 1, 2025, https://huggingface.co/papers/2508.07407
3. Model drift, data leaks and deepfakes: Rethinking AI governance in the age of autonomous risk | IAPP, accessed on October 1, 2025, https://iapp.org/news/a/model-drift-data-leaks-and-deepfakes-rethinking-ai-governance-in-the-age-of-autonomous-risk
4. When AI Goes Astray: Understading Model Drift - ASHP, accessed on October 1, 2025, https://www.ashp.org/-/media/assets/pharmacy-practice/resource-centers/Digital-Health-and-Artificial-Intelligence/docs/When-AI-Goes-Astray.pdf
5. What Is Model Drift? | IBM, accessed on October 1, 2025, https://www.ibm.com/think/topics/model-drift
6. What is concept drift in ML, and how to detect and address it - Evidently AI, accessed on October 1, 2025, https://www.evidentlyai.com/ml-in-production/concept-drift
7. Concept drift - Wikipedia, accessed on October 1, 2025, https://en.wikipedia.org/wiki/Concept_drift
8. Model Drift: What It Is & How To Avoid Drift in AI/ML Models - Splunk, accessed on October 1, 2025, https://www.splunk.com/en_us/blog/learn/model-drift.html
9. Concept Drift vs Data Drift - How Does AI Embrace it all? - USDSI, accessed on October 1, 2025, https://www.usdsi.org/data-science-insights/concept-drift-vs-data-drift-how-does-ai-embrace-it-all
10. PointGuard AI Expands Protection to AI Agents - Security Boulevard, accessed on October 1, 2025, https://securityboulevard.com/2025/09/pointguard-ai-expands-protection-to-ai-agents/
11. Fully Autonomous AI Agents Should Not be Developed - arXiv, accessed on October 1, 2025, https://arxiv.org/pdf/2502.02649
12. The AI Supply Chain: Lessons from the Drift Incident - Acuvity, accessed on October 1, 2025, https://acuvity.ai/the-ai-supply-chain-lessons-from-the-drift-incident/

13. Agentic security unlocked: How enterprises can safeguard autonomous AI Agents, accessed on October 1, 2025, https://www.youtube.com/watch?v=MJWLiwG0CSw
14. Contain or be contained: The security imperative of controlling autonomous AI | CyberScoop, accessed on October 1, 2025, https://cyberscoop.com/security-automonous-ai-threat-response/
15. Keep your site safe and secure – WordPress.com Support, accessed on October 1, 2025, https://wordpress.com/support/security/
16. WordPress Security Services: Scan, Fix & Protect | SiteLock, accessed on October 1, 2025, https://www.sitelock.com/wordpress-security/
17. Wordfence: WordPress Security Plugin, accessed on October 1, 2025, https://www.wordfence.com/
18. Wordfence Security – Firewall, Malware Scan, and Login Security - WordPress.org, accessed on October 1, 2025, https://wordpress.org/plugins/wordfence/
19. 5 Best WordPress Security Plugins Compared | Sucuri, accessed on October 1, 2025, https://sucuri.net/comparison/best-wordpress-security-plugin/
20. 5 Best WordPress Security Plugins to Protect Your Site in 2024 - GeeksforGeeks, accessed on October 1, 2025, https://www.geeksforgeeks.org/wordpress/best-wordpress-security-plugins/
21. What is a Web Application Firewall (WAF) for WordPress? - Jetpack, accessed on October 1, 2025, https://jetpack.com/resources/wordpress-web-application-firewall/
22. Ontology-Oriented Software Development | Palantir | Palantir Blog, accessed on October 1, 2025, https://blog.palantir.com/ontology-oriented-software-development-68d7353fdb12
23. [CyberSec] Semantic and Ontology in Cybersecurity | by Pietro Romano / SecBeret | Medium, accessed on October 1, 2025, https://medium.com/@tribal.secberet/cybersec-semantic-and-ontology-in-cybersecurity-442646ae0c36
24. Ontology-Driven Cybersecurity Learning Factory: A Use Case for Securing Electrical Company Networks, accessed on October 1, 2025, https://www.utwente.nl/en/eemcs/fois2024/resources/papers/dawood-marengwa-ontology-driven-cybersecurity-learning-factory-a-use-case-for-securing-electricity-utilities.pdf
25. Cybersecurity Ontologies: A Systematic Literature Review - Redalyc, accessed on October 1, 2025, https://www.redalyc.org/journal/5122/512267931002/html/
26. Developing an Ontology for Cyber Security Knowledge Graphs - OSTI, accessed on October 1, 2025, https://www.osti.gov/servlets/purl/1424501
27. An Ontology-Based Cybersecurity Framework for AI-Enabled ... - MDPI, accessed on October 1, 2025, https://www.mdpi.com/1999-5903/16/3/69
28. GraphGen4Code | A Toolkit for Generating Code Knowledge Graphs, accessed on October 1, 2025, https://wala.github.io/graph4code/
29. Codebase to Knowledge Graph generator : r/LocalLLaMA - Reddit, accessed on

October 1, 2025, https://www.reddit.com/r/LocalLLaMA/comments/1mzvk44/codebase_to_knowledge_graph_generator/
30. What are Affordances? | IxDF, accessed on October 1, 2025, https://www.interaction-design.org/literature/topics/affordances
31. Affordance - Wikipedia, accessed on October 1, 2025, https://en.wikipedia.org/wiki/Affordance
32. A Review of Application of Affordance Theory in Information Systems - Scirp.org, accessed on October 1, 2025, https://www.scirp.org/pdf/jssm_2018020814342288.pdf
33. (PDF) Affordances and their ontological core - ResearchGate, accessed on October 1, 2025, https://www.researchgate.net/publication/360347199_Affordances_and_their_ontological_core
34. Affordance Ontology: Towards a Unified Description of Affordances as Events - Central Washington University, accessed on October 1, 2025, https://digitalcommons.cwu.edu/ijurca/vol8/iss2/4/
35. (PDF) Affordance-based ontology design for ubiquitous robots, accessed on October 1, 2025, https://www.researchgate.net/publication/4363425_Affordance-based_ontology_design_for_ubiquitous_robots
36. A Formal Model of Affordances for Flexible Robotic Task Execution - Institute for Artificial Intelligence - Uni Bremen, accessed on October 1, 2025, https://ai.uni-bremen.de/papers/bessler20affordances.pdf
37. A survey of ontology-enabled processes for dependable robot autonomy - Frontiers, accessed on October 1, 2025, https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2024.1377897/full
38. Self-Evolving Agents: The Ultimate Frontier in Artificial Intelligence | by noailabs - Medium, accessed on October 1, 2025, https://noailabs.medium.com/self-evolving-agents-the-ultimate-frontier-in-artificial-intelligence-7b7842c1f472
39. self-evolution with language feed - arXiv, accessed on October 1, 2025, https://arxiv.org/pdf/2310.0533
40. EvoAgentX/Awesome-Self-Evolving-Agents - GitHub, accessed on October 1, 2025, https://github.com/EvoAgentX/Awesome-Self-Evolving-Agents
41. A Survey of Self-Evolving Agents: On Path to Artificial Super Intelligence | alphaXiv, accessed on October 1, 2025, https://www.alphaxiv.org/overview/2507.21046
42. A Comprehensive Survey of Self-Evolving AI Agents [pdf] - Hacker News, accessed on October 1, 2025, https://news.ycombinator.com/item?id=44884091
43. (PDF) A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems - ResearchGate, accessed on October 1, 2025, https://www.researchgate.net/publication/394439110_A_Comprehensive_Survey_

of_Self-Evolving_AI_Agents_A_New_Paradigm_Bridging_Foundation_Models_and_Lifelong_Agentic_Systems
44. Self Evolving AI Agents - Cobus Greyling - Medium, accessed on October 1, 2025, https://cobusgreyling.medium.com/self-evolving-ai-agents-12304ee7912e
45. A Comprehensive Survey of Self-Evolving AI Agents (August 2025) - YouTube, accessed on October 1, 2025, https://www.youtube.com/watch?v=wLXaL_58hNQ
46. Evaluating LLMs using semantic entropy | Thoughtworks United States, accessed on October 1, 2025, https://www.thoughtworks.com/en-us/insights/blog/generative-ai/Evaluating-LLM-using-semantic-entropy
47. Information theory - Wikipedia, accessed on October 1, 2025, https://en.wikipedia.org/wiki/Information_theory
48. The Role of Semantics in Agentic AI: Enabling Context, Intent, and ..., accessed on October 1, 2025, https://www.arionresearch.com/blog/hpaddo9fvkz6arupd85ptth89d16ij
49. What is Agentic AI? | IBM, accessed on October 1, 2025, https://www.ibm.com/think/topics/agentic-ai
50. Core concepts of Agentic AI and AI agents - Ready Tensor, accessed on October 1, 2025, https://app.readytensor.ai/publications/core-concepts-of-agentic-ai-and-ai-agents-82IYI7TWVtvP
51. (PDF) Semantic Clustering of Functional Requirements Using ..., accessed on October 1, 2025, https://www.researchgate.net/publication/327410624_Semantic_Clustering_of_Functional_Requirements_Using_Agglomerative_Hierarchical_Clustering
52. Guide to the Types of Access Control Models | NordLayer Learn, accessed on October 1, 2025, https://nordlayer.com/learn/access-control/types-of-access-control/
53. What Is Access Control? - Network Cybersecurity Systems - Fortinet, accessed on October 1, 2025, https://www.fortinet.com/resources/cyberglossary/access-control
54. An Overview of Security Breach Probability Models - MDPI, accessed on October 1, 2025, https://www.mdpi.com/2227-9091/10/11/220
55. (PDF) Probability models for access security system architectures - ResearchGate, accessed on October 1, 2025, https://www.researchgate.net/publication/317172996_Probability_models_for_access_security_system_architectures
56. Dynamic risk-based access control model | Download Scientific ..., accessed on October 1, 2025, https://www.researchgate.net/figure/Dynamic-risk-based-access-control-model_fig2_364152745
57. Risk-Adaptive Access Control (RADAC): A Deep Dive | Teleport, accessed on October 1, 2025, https://goteleport.com/learn/risk-adaptive-access-control/
58. What is Dynamic Access Control (DAC)? Full 2025 Guide - Rippling, accessed on October 1, 2025, https://www.rippling.com/blog/dynamic-access-control

59. Access Control Models Explained - tenfold, accessed on October 1, 2025, https://www.tenfold-security.com/en/wiki/access-control-models/
60. rEGGression: an Interactive and Agnostic Tool for the Exploration of Symbolic Regression Models - arXiv, accessed on October 1, 2025, https://arxiv.org/html/2501.17859v1
61. Symbolic Regression by Exhaustive Search: Reducing the Search Space Using Syntactical Constraints and Efficient Semantic Structure Deduplication - ResearchGate, accessed on October 1, 2025, https://www.researchgate.net/publication/341234999_Symbolic_Regression_by_Exhaustive_Search_Reducing_the_Search_Space_Using_Syntactical_Constraints_and_Efficient_Semantic_Structure_Deduplication
62. (PDF) Assessing the Efficacy of Symbolic Regression for Scientific Data Classification and Regression - ResearchGate, accessed on October 1, 2025, https://www.researchgate.net/publication/375880291_Assessing_the_Efficacy_of_Symbolic_Regression_for_Scientific_Data_Classification_and_Regression
63. AEG: Automatic Exploit Generation - Computer Science, accessed on October 1, 2025, https://cseweb.ucsd.edu/~dstefan/cse227-spring21/papers/avgerinos:aeg.pdf
64. automatic exploit Generation, accessed on October 1, 2025, https://zzm7000.github.io/teaching/2021springcse703/papers/2560217.2560219.pdf
65. A Systematic Literature Review on Automated Exploit and Security Test Generation - arXiv, accessed on October 1, 2025, https://arxiv.org/html/2502.04953v1
66. A Systematic Literature Review on Automated Exploitand Security Test Generation - arXiv, accessed on October 1, 2025, https://arxiv.org/pdf/2502.04953
67. Neuro-Symbolic AI for Cybersecurity: State of the Art, Challenges, and Opportunities - arXiv, accessed on October 1, 2025, https://arxiv.org/pdf/2509.06921
68. Neurosymbolic AI & Cyber Reasoning: The Future of Cybersecurity ..., accessed on October 1, 2025, https://blog.rsisecurity.com/neurosymbolic-ai-advanced-cyber-reasoning-the-future-of-smarter-cybersecurity/
69. Neuro-Symbolic AI for Cybersecurity: State of the Art, Challenges, and Opportunities - arXiv, accessed on October 1, 2025, https://arxiv.org/abs/2509.06921
70. Experimenting with neurosymbolic AI for defending against cyber attacks, accessed on October 1, 2025, https://neurosymbolic-ai-journal.com/system/files/nai-paper-828.pdf
71. What is Zero Trust? - Guide to Zero Trust Security - CrowdStrike.com, accessed on October 1, 2025, https://www.crowdstrike.com/en-us/cybersecurity-101/zero-trust-security/
72. Zero Trust Architecture - Sepio Cyber, accessed on October 1, 2025, https://sepiocyber.com/blog/zero-trust-architecture/
73. What Is Zero Trust Architecture? Key Elements and Use Cases - Palo Alto

Networks, accessed on October 1, 2025, https://www.paloaltonetworks.com/cyberpedia/what-is-a-zero-trust-architecture
74. Zero Trust security | What is a Zero Trust network? | Cloudflare, accessed on October 1, 2025, https://www.cloudflare.com/learning/security/glossary/what-is-zero-trust/
75. Continuous Authentication: The Core of Zero Trust Architecture - swIDch, accessed on October 1, 2025, https://www.swidch.com/resources/blogs/why-should-continuous-authentication-be-at-the-heart-of-your-zero-trust-architecture